



IIC2343 - Arquitectura de Computadores (II/2026)

## Ayudantía 4: Programabilidad y Computador básico

Ayudantes: Ignacio Gajardo (gajardo.ignacio@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl),  
Nicolás Romo Aubele (nroma@uc.cl), José Mendoza (jfmendoza@uc.cl)

### Pregunta 1: Preguntas conceptuales y Análisis de código

- Describa y entregue las señales de una instrucción que es soportada por el computador básico, pero que no se encuentra en el conjunto de instrucciones de este (*i.e.* que no necesita modificaciones de la arquitectura para implementarla).
- ¿Por qué motivo no está soportada la instrucción `ADD B, LIT` en el computador básico visto en clases? Explique con palabras cómo podría implementarse.
- Describa qué hace el siguiente fragmento de código, escrito en el *assembly* del computador básico del curso. Indique los valores de los registros A y B, y el de las variables al finalizar la ejecución del código. Recuerde que el computador básico del curso realiza operaciones con 8 bits.

**Bonus:** Adicionalmente, indique los valores finales de las *flags* C, Z y N.

```
DATA:
    var1 4
    var2 8
    var3 0

CODE:
    MOV A, (var1)
    SHR B,A
    ADD B,A
    MOV (var1), B
    MOV A, (var2)
    SHL B,A
    ADD B,A
    MOV (var2), B
    MOV A, (var1)
    AND A, B
    NOT A, A
    MOV (var3), A
    INC (var3)
```

### Solución:

a. Como se puede observar, en la conjunto de instrucciones del computador básico hay un gran número de *opcodes* que no se están utilizando, en el caso del computador básico sin saltos las instrucciones son de 7 *bits*, lo que implica que tenemos un gran número de combinación de señales que no estamos utilizando.

Un ejemplo es una instrucción que suma los valores de los registros de A y LIT, pero que guarde en ambos registros a la vez, esto implica que las dos señales de carga de los registros deben estar activas, las señales de esta instrucción son las siguiente:

| Instrucción | Operandos | Condición | L <sub>A</sub> | L <sub>B</sub> | S <sub>A</sub> 0,1 | S <sub>B</sub> 0,1 | S <sub>Op</sub> 0,1,2 | S <sub>Add</sub> 0,1 | W |
|-------------|-----------|-----------|----------------|----------------|--------------------|--------------------|-----------------------|----------------------|---|
| ADD         | A,B       |           | 1              | 1              | A                  | B                  | ADD                   | -                    | 0 |

b. Las instrucciones B, LIT no se puede implementar para cualquier literal debido a que en la arquitectura actual, no existe una conexión entre el MUX A y la memoria de instrucciones, esto implica que la ALU no puede recibir el valor del registro B y de un literal de la memoria de instrucciones a la vez, los dos únicos literales que pueden operar con B son 1y 0 que son las conexiones que están en el MUX A. Para poder implementar la instrucción es importante notar que se debe modificar la arquitectura agregando una conexión entre la memoria de instrucciones y el espacio libre del MUX A, por lo que, haciendo esto y asignándole la señal de selección libre se podría ejecutar esta instrucción en el computador.

c. A continuación, se muestra el valor que cambia después de cada instrucción:

DATA:

```
var1 4
var2 8
var3 0
```

CODE:

```
MOV A, (var1) // A      = 00000100
SHR B,A       // B      = 00000010
ADD B,A       // B      = 00000110
MOV (var1), B // (var1) = 00000110
MOV A, (var2) // A      = 00001000
SHL B,A       // B      = 00010000
ADD B,A       // B      = 00011000
MOV (var2), B // (var2) = 00011000
MOV A, (var1) // A      = 00000110
AND A, B      // A      = 00000000
NOT A, A      // A      = 11111111
MOV (var3), A // (var3) = 11111111
INC (var3)    // (var3) = 00000000
```

El fragmento de código realiza operaciones de desplazamiento y suma sobre los valores iniciales de las variables. Dado que `var1 = 4`, se carga el valor de `var1` en A. Primero, con la instrucción `SHR B, A`, se realiza un corrimiento a la derecha de una posición sobre el valor de A, es decir, `SHR B, A = 2`, y se guarda en el registro B. Luego se ejecuta `ADD B, A`, con lo cual `B = 4 + 2 = 6`. Este resultado se almacena en `var1`, por lo que ahora `var1 = 6`.

Posteriormente, se carga el valor de `var2 = 8` en A y se aplica `SHL B, A`, es decir, un corrimiento

a la izquierda de una posición: **SHL B, A** = 16, que se almacena en el registro B. A continuación, con la instrucción **ADD B, A**, se obtiene  $B = 16 + 8 = 24$ , valor que se almacena en **var2**.

Luego, se carga el valor actualizado de **var1** en el registro A y se ejecuta **AND A, B** = 1, resultando  $A = 6 \text{ AND } 24 = 0$ . A este valor se le aplica la operación **NOT A**, obteniendo  $A = 255$ . Este resultado se almacena en **var3** y se termina incrementando en 1, obteniendo **var3** = 0. Esto debido a que se está operando con 8 bits.

Finalmente, después de estas operaciones, los registros y variables quedan con los valores:

**Reg A:** 11110b = 30d

**Reg B:** 11000b = 24d

**var1:** 00110b = 6d

**var2:** 11000b = 24d

**var3:** 11111b = 31d

**Bonus:** Los valores de las *flags* son  $C = 1$ ,  $Z = 1$  y  $N = 0$ .

## Pregunta 2: Modificaciones al computador básico I

Modifique la microarquitectura de la CPU Básica para poder realizar lo siguiente:

- Direccionamiento indirecto por registro base + *offset*. Por ejemplo, `MOV A, (B + offset)`.
- Además del direccionamiento anterior, agregar direccionamiento indirecto por registro base + registro índice. Por ejemplo, `MOV A, (B + A)`.
- Condition code* I, que indica cuando un número es impar. Indique cómo se debe procesar el *output* de la ALU para obtener este *condition code*. Este *condition code* debe incluirse en el *Status Register*.

Para cada uno de estos casos se deben señalar los bits de control, buses, componentes digitales y cualquier otra modificación que estime pertinente.

### Solución:

- Para añadir una nueva forma de direccionamiento, se agrega una nueva entrada al Mux Address de la siguiente forma (la modificación de este ítem se muestra con azul):

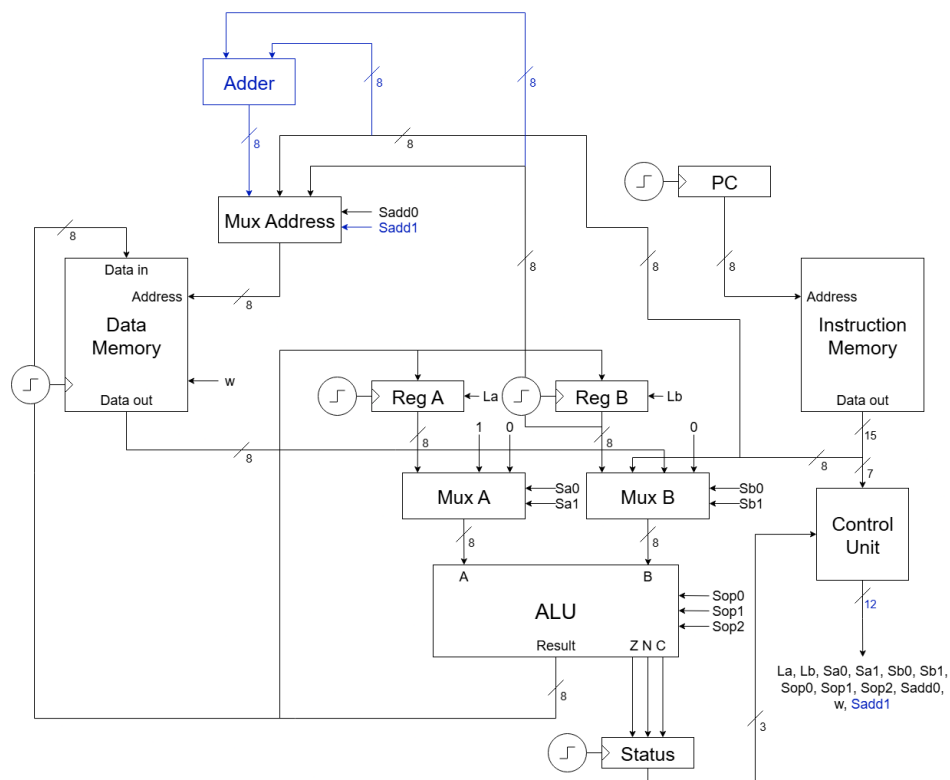


Figura 1: Diagrama Solución a.

(b) De manera similar al item anterior, se agrega una nueva entrada al Mux Address de la siguiente forma (la modificación de este item se muestra con rojo):

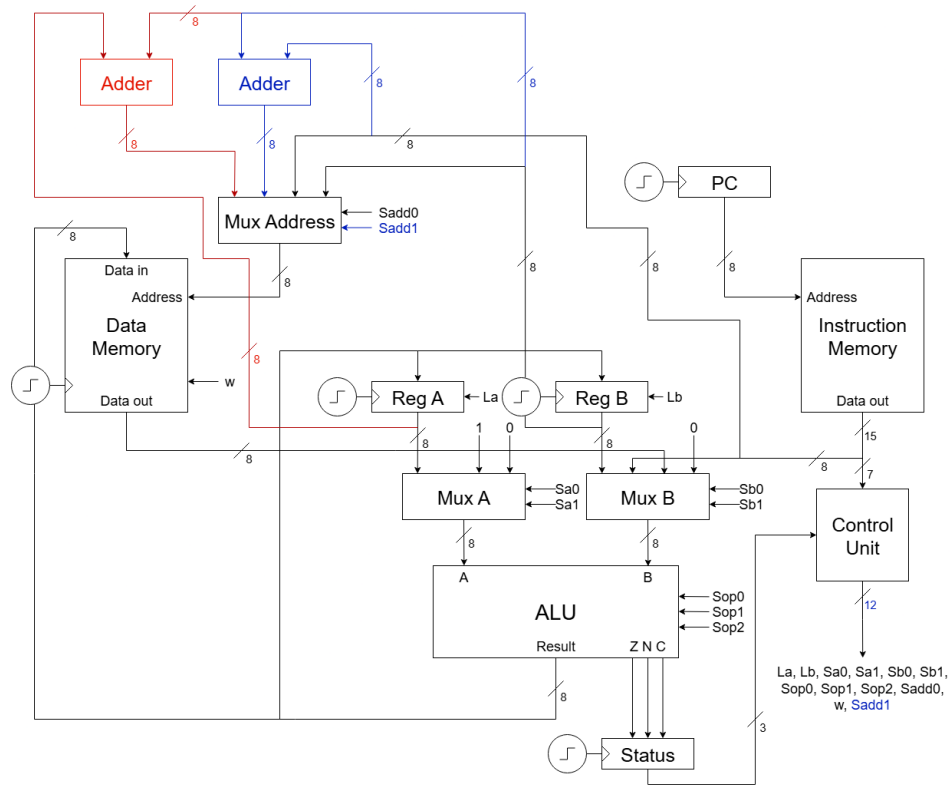


Figura 2: Diagrama Solución b.

- (c) Para esta modificación, basta con conectar el bit menos significativo de Result al registro Status. De este modo, si el bit es 1, el resultado será impar; en caso contrario, será par (la modificación de este item se muestra con magenta):

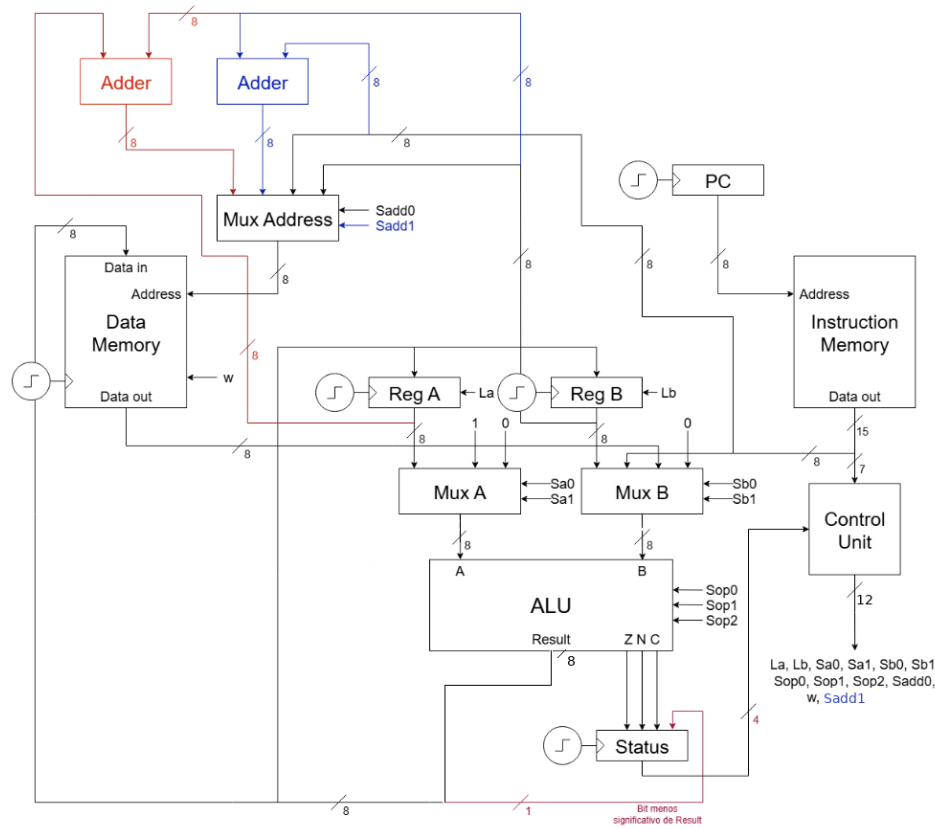


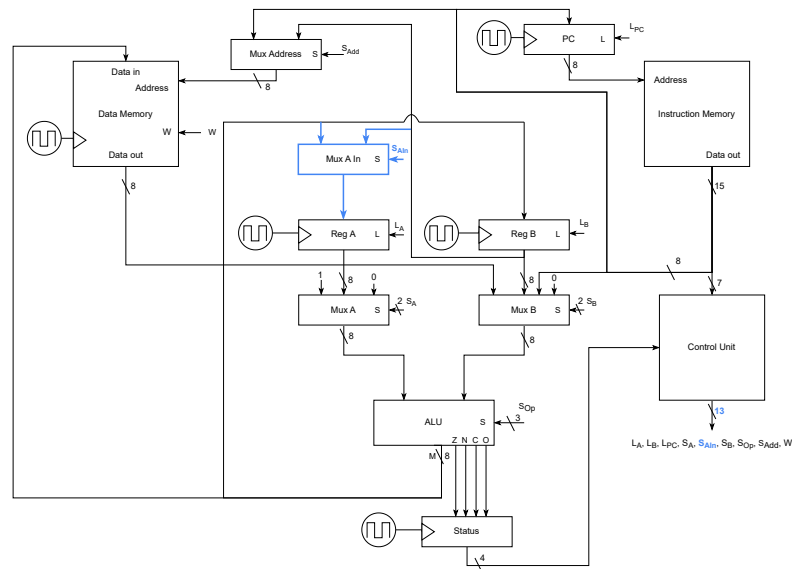
Figura 3: Diagrama Solución c.

### Pregunta 3: Modificaciones al computador básico II (I1 2025-2)

En los ítems (a) y (b), se le describirán instrucciones que deberá implementar en el computador básico a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida (“-” si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo.**

- (a) **SWAP A,B**. Almacena en el registro A el valor del registro B; y almacena en el registro B el valor del registro A, *i.e.* intercambia sus valores.
- (b) **JGTCMP A,B,label**. Salta a la dirección asociada a **label** si, y solo si las *flags* Z y N provenientes del registro **Status** son iguales a 0 y, al mismo tiempo, ejecuta la operación  $A - B$  en la ALU sin actualizar los registros A y B, independiente de si hay salto o no.

**Solución:** A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se añade el multiplexor **Mux A In** y su señal de selección  $S_{AIn}$  para seleccionar el valor de entrada del registro A. Para la instrucción **SWAP A,B**, se seleccionará el valor del registro B; en cualquier otro caso, se seleccionará el valor proveniente de la ALU. Cabe destacar que se pudo haber implementado un cambio similar para el registro B, pero no es necesario para implementar la instrucción.
- Para la instrucción **JGTCMP A,B,label**, no se necesitan modificaciones.

A continuación, la tabla de señales resultante:

| Instrucción      | L <sub>A</sub> | L <sub>B</sub> | L <sub>PC</sub> | W | S <sub>A</sub> | S <sub>B</sub> | S <sub>OP</sub> | S <sub>Add</sub> | S <sub>AIn</sub> |
|------------------|----------------|----------------|-----------------|---|----------------|----------------|-----------------|------------------|------------------|
| SWAP A,B         | 1              | 1              | 0               | 0 | A              | ZERO           | ADD             | -                | B                |
| JGTCMP A,B,label | 0              | 0              | Z = 0 & N = 0   | 0 | A              | B              | SUB             | -                | -                |

- (c) A continuación, se adjuntan dos fragmentos de código (1) y (2) escritos en el Assembly del computador básico del curso, con la diferencia de que (2) incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros A y B al finalizar la ejecución del código.

(1)

```
DATA:
    var1 -1
    var2 1
    var3 -1
    var4 1
CODE:
    MOV A,125 ; 125 = 01111101b
    NOT A,A
    SHL A,A
    SHR A,A
    MOV B,A
    MOV A,(B)
    JGT case_1
    JMP case_2
case_1:
    MOV A,-1
    JMP end
case_2:
    MOV A,1
end:
```

(2)

```
DATA:
    var1 5
    var2 16
CODE:
    init:
        MOV A,(var1)
        MOV B,(var2)
        CMP A,B
        JGTCMP A,B,case_1
        JLT case_2
    case_1:
        JMP continue
    case_2:
        SWAP A,B
    continue:
        SWAP A,B
        NOT A,A
        SWAP A,B
        INC B
        ADD A,B
```



**Solución:** El fragmento de código (1) accede, a través de direccionamiento indirecto, a una de las variables definidas en DATA. Si el valor leído es positivo, se guarda en el registro A un -1; en otro caso, se guarda un 1. El fragmento de código (2), por otra parte, computa el valor absoluto de la resta entre var1 y var2 y guarda el resultado en el registro A. A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

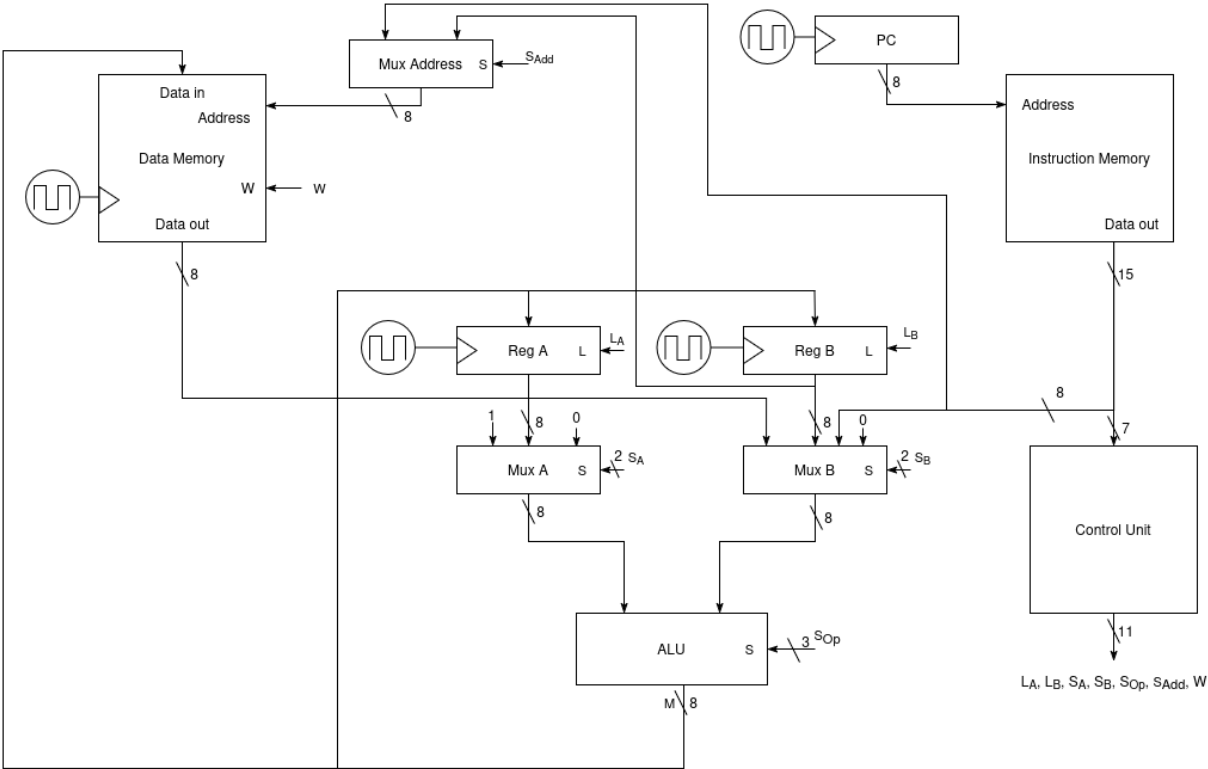
```
DATA:
var1 -1      ; MEM[0] = -1
var2 1       ; MEM[1] = 1
var3 -1      ; MEM[2] = -1
var4 1       ; MEM[3] = 1
CODE:
MOV A,125    ; A = 01111101b = 125
NOT A,A      ; A = 10000010b = 130
SHL A,A      ; A = 00000100b = 4
SHR A,A      ; A = 00000010b = 2
MOV B,A      ; B = 00000010b = 2
MOV A,(B)    ; A = MEM[2] = -1
JGT case_1   ; N = 1 -> No salta
JMP case_2   ; Salta a case_2
case_1:
MOV A,-1     ; No se ejecuta
JMP end      ; No se ejecuta
case_2:
MOV A,1      ; A = 1
end:         ; A = 1, B = 2
```

(2)

```
DATA:
var1 5                ; MEM[0] = 5
var2 16               ; MEM[1] = 16
CODE:
init:
MOV A,(var1)          ; A = MEM[0] = 5
MOV B,(var2)          ; B = MEM[1] = 16
CMP A,B               ; A - B -> Z = 0, N = 1
JGTCMP A,B,case_1     ; N = 1 -> No salta, A - B -> Z = 0, N = 1
JLT case_2            ; N = 1 -> Salta a case_2
case_1:
JMP continue         ; No se ejecuta
case_2:
SWAP A,B              ; A = 16, B = 5
continue:
SWAP A,B              ; A = 5, B = 16
NOT A,A               ; A = NOT(5)
SWAP A,B              ; A = 16, B = NOT(5)
INC B                 ; A = 16, B = NOT(5)+1 = C2(5) = -5
ADD A,B               ; A = 11, B = -5
```

Para cada fragmento de código, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.

# Computador básico sin saltos



## Tabla de instrucciones

| Instrucción | Operandos | Opcode  | Condición | L <sub>A</sub> | L <sub>B</sub> | S <sub>A</sub> 0,1 | S <sub>B</sub> 0,1 | S <sub>Op</sub> 0,1,2 | S <sub>Add</sub> 0,1 | W |
|-------------|-----------|---------|-----------|----------------|----------------|--------------------|--------------------|-----------------------|----------------------|---|
| MOV         | A,B       | 0000000 |           | 1              | 0              | ZERO               | B                  | ADD                   | -                    | 0 |
|             | B,A       | 0000001 |           | 0              | 1              | A                  | ZERO               | ADD                   | -                    | 0 |
|             | A,Lit     | 0000010 |           | 1              | 0              | ZERO               | LIT                | ADD                   | -                    | 0 |
|             | B,Lit     | 0000011 |           | 0              | 1              | ZERO               | LIT                | ADD                   | -                    | 0 |
|             | A,(Dir)   | 0000100 |           | 1              | 0              | ZERO               | DOUT               | ADD                   | LIT                  | 0 |
|             | B,(Dir)   | 0000101 |           | 0              | 1              | ZERO               | DOUT               | ADD                   | LIT                  | 0 |
|             | (Dir),A   | 0000110 |           | 0              | 0              | A                  | ZERO               | ADD                   | LIT                  | 1 |
|             | (Dir),B   | 0000111 |           | 0              | 0              | ZERO               | B                  | ADD                   | LIT                  | 1 |
|             | A,(B)     | 0001000 |           | 1              | 0              | ZERO               | DOUT               | ADD                   | B                    | 0 |
| ADD         | B,(B)     | 0001001 |           | 0              | 1              | ZERO               | DOUT               | ADD                   | B                    | 0 |
|             | (B),A     | 0001010 |           | 1              | 0              | A                  | ZERO               | ADD                   | B                    | 1 |
|             | A,B       | 0001011 |           | 1              | 0              | A                  | B                  | ADD                   | -                    | 0 |
|             | B,A       | 0001100 |           | 0              | 1              | A                  | B                  | ADD                   | -                    | 0 |
|             | A,Lit     | 0001101 |           | 1              | 0              | A                  | LIT                | ADD                   | -                    | 0 |
|             | A,(Dir)   | 0001110 |           | 1              | 0              | A                  | DOUT               | ADD                   | LIT                  | 0 |
|             | A,(B)     | 0001111 |           | 1              | 0              | A                  | DOUT               | ADD                   | B                    | 0 |
|             | (Dir)     | 0010000 |           | 0              | 0              | A                  | B                  | ADD                   | LIT                  | 1 |
|             | (Dir)     | 0010000 |           | 0              | 0              | A                  | B                  | ADD                   | LIT                  | 1 |
| SUB         | A,B       | 0010001 |           | 1              | 0              | A                  | B                  | SUB                   | -                    | 0 |
|             | B,A       | 0010010 |           | 0              | 1              | A                  | B                  | SUB                   | -                    | 0 |
|             | A,Lit     | 0010011 |           | 1              | 0              | A                  | LIT                | SUB                   | -                    | 0 |
|             | A,(Dir)   | 0010100 |           | 1              | 0              | A                  | DOUT               | SUB                   | LIT                  | 0 |
|             | A,(B)     | 0010101 |           | 1              | 0              | A                  | DOUT               | SUB                   | B                    | 0 |
|             | (Dir)     | 0010110 |           | 0              | 0              | A                  | B                  | SUB                   | LIT                  | 1 |
|             | A,B       | 0010111 |           | 1              | 0              | A                  | B                  | AND                   | -                    | 0 |
|             | B,A       | 0011000 |           | 0              | 1              | A                  | B                  | AND                   | -                    | 0 |
|             | A,Lit     | 0011001 |           | 1              | 0              | A                  | LIT                | AND                   | -                    | 0 |
| AND         | A,(Dir)   | 0011010 |           | 1              | 0              | A                  | DOUT               | AND                   | LIT                  | 0 |
|             | A,(B)     | 0011011 |           | 1              | 0              | A                  | DOUT               | AND                   | B                    | 0 |
|             | (Dir)     | 0011100 |           | 0              | 0              | A                  | B                  | AND                   | LIT                  | 1 |
|             | A,B       | 0011101 |           | 1              | 0              | A                  | B                  | OR                    | -                    | 0 |
|             | B,A       | 0011110 |           | 0              | 1              | A                  | B                  | OR                    | -                    | 0 |
|             | A,Lit     | 0011111 |           | 1              | 0              | A                  | LIT                | OR                    | -                    | 0 |
|             | A,(Dir)   | 0100000 |           | 1              | 0              | A                  | DOUT               | OR                    | LIT                  | 0 |
|             | A,(B)     | 0100001 |           | 1              | 0              | A                  | DOUT               | OR                    | B                    | 0 |
|             | (Dir)     | 0100010 |           | 0              | 0              | A                  | B                  | OR                    | LIT                  | 1 |
| OR          | A,A       | 0100011 |           | 1              | 0              | A                  | -                  | NOT                   | -                    | 0 |
|             | B,A       | 0100100 |           | 0              | 1              | A                  | -                  | NOT                   | -                    | 0 |
|             | (Dir)     | 0100101 |           | 0              | 0              | A                  | B                  | NOT                   | LIT                  | 1 |
|             | A,B       | 0100110 |           | 1              | 0              | A                  | B                  | XOR                   | -                    | 0 |
|             | B,A       | 0100111 |           | 0              | 1              | A                  | B                  | XOR                   | -                    | 0 |
|             | A,Lit     | 0101000 |           | 1              | 0              | A                  | LIT                | XOR                   | -                    | 0 |
|             | A,(Dir)   | 0101001 |           | 1              | 0              | A                  | DOUT               | XOR                   | LIT                  | 0 |
|             | A,(B)     | 0101010 |           | 1              | 0              | A                  | DOUT               | XOR                   | B                    | 0 |
|             | (Dir)     | 0101011 |           | 0              | 0              | A                  | B                  | XOR                   | LIT                  | 1 |
| XOR         | A,A       | 0101100 |           | 1              | 0              | A                  | -                  | SHL                   | -                    | 0 |
|             | B,A       | 0101101 |           | 0              | 1              | A                  | -                  | SHL                   | -                    | 0 |
|             | (Dir)     | 0101110 |           | 0              | 0              | A                  | B                  | SHL                   | LIT                  | 1 |
|             | A,A       | 0101111 |           | 1              | 0              | A                  | -                  | SHR                   | -                    | 0 |
|             | B,A       | 0110000 |           | 0              | 1              | A                  | -                  | SHR                   | -                    | 0 |
|             | (Dir)     | 0110001 |           | 0              | 0              | A                  | B                  | SHR                   | LIT                  | 1 |
|             | B         | 0110010 |           | 0              | 1              | ONE                | B                  | ADD                   | -                    | 0 |
|             | (B)       | 0110011 |           | 0              | 0              | ONE                | DOUT               | ADD                   | B                    | 1 |
|             | (Dir)     | 0110100 |           | 0              | 0              | ONE                | DOUT               | ADD                   | LIT                  | 1 |

Tabla 1: ISA del computador básico.

## Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

